

Java DataBase Connectivity



Dr Khadim DRAME*

kdrame@univ-zig.sn

20 décembre 2022

Table des matières

1	Introduction	1
2	Connexion à une base de données	2
3	Manipulation de requêtes SQL	3
3.1	Manipulation de requêtes SQL	3
3.2	Manipulation de requêtes SQL paramétrées	4
4	Manipulation de résultats de requêtes SQL	5

1 Introduction

Java DataBase Connectivity (JDBC) est une API Java pour interagir avec les bases de données relationnelles (BDR). Elle permet d'accéder aux bases de données et de les manipuler en utilisant le langage SQL ; elle supporte les différents types de requêtes SQL. JDBC est indépendant de tout système de gestion de bases de données (SGBD).

Le package `java.sql` fournit un ensemble d'interfaces pour réaliser différentes opérations sur les bases de données : `Connection`, `Statement`, `PreparedStatement`, `ResultSet`, etc. Les implémentations de ces interfaces sont assurées par des pilotes (drivers) spécifiques aux SGBDs. La plupart des SGBDs disposent de pilotes JDBC, fournis par leurs éditeurs.

*UFR Sciences et Technologies, Université Assane Seck de Ziguinchor

TABLE 1 – Formats de l'URL selon le SGBD

SGBD	Format de l'URL de connexion
MySQL	<code>jdbc:mysql://[hote]:[port]/[nombd]</code>
PostgreSQL	<code>jdbc:postgresql://[hote]:[port]/[nombd]</code>
Oracle DB	<code>jdbc:oracle:thin:@[hote]:[port]/[nombd]</code>
MariaDB	<code>jdbc:mariadb://[hote]:[port]/[nombd]</code>

2 Connexion à une base de données

Pour interagir avec une base de données, il faut nécessairement un pilote JDBC spécifique au SGBD utilisé. Ce pilote est fourni sous la forme d'un fichier jar (fichier à intégrer dans la *classpath*).

Pour établir une connexion à une base de données, on utilise une des méthodes **statiques** `getConnection()` de la classe `DriverManager`. Ces méthodes prennent en paramètres l'URL de la base de données et éventuellement d'autres informations (nom d'utilisateur, mot de passe) et renvoient une instance de `Connection` ou `null` si le driver ne convient pas. Le format de l'URL de connexion dépend du SGBD utilisé (Cf Table 1).

Dans certaines versions de SGBDs (par exemple, MySQL 8 et les versions plus récentes), il est nécessaire de spécifier le fuseau horaire à utiliser par le serveur en passant le paramètre `serverTimezone` dans l'URL de connexion. Exemple :

`jdbc:mysql://localhost:3306/club?serverTimezone=GMT`

Lorsque la connexion n'est plus nécessaire, il faut la fermer pour libérer les ressources allouées.

```

1 String url = "jdbc:mysql://localhost:3306/club?serverTimezone=GMT";
2 //String url = "jdbc:postgresql://localhost:5432/club";
3 String user = "root";
4 String password = "";
5 Connection connexion = null;
6 try {
7     connexion = DriverManager.getConnection(url, user, password);
8     if (connexion != null) {
9         System.out.println("Connexion réussie !");
10    } else {
11        System.out.println("Echec de le connexion!");
12    }
13 } catch (SQLException e) {
14     e.printStackTrace();
15 } finally {
16     //fermeture de la connexion
17     if (connexion != null) {
18         connexion.close();
19     }
20 }
```

Listing 1 – Exemple de connexion à une base de données

`Connection` étend l'interface `AutoCloseable`. Donc, elle peut être utilisée dans une instruction **try-with-resources** (Cf Exceptions).

```

1 String url = "jdbc:mysql://localhost:3306/club?serverTimezone=GMT";
2 //String url = "jdbc:postgresql://localhost:5432/club";
3 String user = "root";
4 String password = "";
```

```

5 try (Connection connexion = DriverManager.getConnection(url, user, password)){
6     if (connexion != null) {
7         System.out.println("Connexion réussie !");
8     } else {
9         System.out.println("Echec de le connexion!");
10    }
11 } catch (SQLException e) {
12     e.printStackTrace();
13 }

```

Listing 2 – Exemple de connexion à une base de données avec **try-with-resources**

3 Manipulation de requêtes SQL

L'interface `Connection` fournit des méthodes permettant de créer des requêtes SQL (statements). Dans l'API JDBC, il y a trois interfaces pour créer de requêtes SQL : `Statement` pour les requêtes simples, `PreparedStatement` pour les requêtes paramétrées et `CallableStatement` pour les procédures stockées.

3.1 Manipulation de requêtes SQL

Les méthodes `createStatement()` de l'interface `Connection` permettent de créer des requête SQL de type `Statement`. L'interface `Statement` dispose de méthodes pour exécuter différents types de requêtes :

- `executeQuery()` : pour exécuter des requêtes de consultation (**select**). Cette méthode renvoie un objet de type `ResultSet` ;
- `executeUpdate()` : pour exécuter des requêtes de mise à jour des données (**insert**, **update**, **delete**) mais également de type DDL (**create table**, **drop table**, etc.). Cette méthode renvoie un entier indiquant le nombre de lignes mises à jour ;
- `execute()` : pour exécuter tout type de requêtes SQL. Elle renvoie un booléen.

`Statement` étant une ressource, elle doit être fermée lorsqu'elle n'est plus utilisée.

Remarque : Puisque `Statement` étend l'interface `AutoCloseable`, elle peut donc être utilisée dans une instruction **try-with-resources** (Cf Exceptions).

```

1 String url = "jdbc:mysql://localhost:3306/club?serverTimezone=GMT";
2 //String url = "jdbc:postgresql://localhost:5432/club";
3 String user = "root";
4 String password = "";
5 //établissement d'une connexion à la base de données club
6 try (Connection connexion = DriverManager.getConnection(url, user, password)){
7     //création de requête SQL
8     Statement st = connexion.createStatement();
9     //exécution d'une requête de consultation
10    //retourne toutes les lignes de la table MEMBRE
11    ResultSet rs = st.executeQuery("SELECT * FROM MEMBRE");
12    //exécution d'une requête de type DML
13    //insère une ligne dans la table MEMBRE
14    int nbl = st.executeUpdate("INSERT INTO MEMBRE VALUES ('1001','Modou','Diop',42,'Enseignant','M')");
15 } catch (SQLException e) {
16     e.printStackTrace();
17 } // les ressources Connection, Statement et ResultSet doivent être fermées

```

Listing 3 – Exemple de création et d'exécution de requêtes SQL

3.2 Manipulation de requêtes SQL paramétrées

Les méthodes `preparedStatement()` de l'interface `Connection` permettent de créer des requêtes SQL paramétrées, c'est à dire de type `PreparedStatement`. L'interface `PreparedStatement` étend `Statement` et hérite toutes ses méthodes permettant d'exécuter des requêtes. Elle peut également être utilisée dans une instruction `try-with-resources`.

Appelées aussi requêtes "à trous", les requêtes paramétrées peuvent être exécutées avec plusieurs ensembles de valeurs. Les valeurs des paramètres sont spécifiées avec des méthodes `setXXX()` (`XXX` représente le type Java correspondant) : `setByte()`, `setShort()`, `setInt()`, `setLong()`, `setFloat()`, `setDouble()`, `setBigDecimal()`, `setBoolean()`, `setString()`, `setDate()`, etc.

```
1 String url = "jdbc:mysql://localhost:3306/club?serverTimezone=GMT";
2 //String url = "jdbc:postgresql://localhost:5432/club";
3 String user = "root";
4 String password = "";
5 //établissement d'une connexion à la base de données club
6 try (Connection connexion = DriverManager.getConnection(url, user, password)){
7     //création et exécution d'une requête de consultation
8     String req = "SELECT * FROM MEMBRE WHERE prenom=? AND age>?";
9     PreparedStatement pst = con.prepareStatement(req);
10    pst.setString(1, "Moustapha");
11    pst.setInt(2, 35);
12    ResultSet rs = pst.executeQuery();
13    //création et exécution d'une requête de type DML
14    String req2 = "INSERT INTO MEMBRE (nom, prenom) VALUES (?,?)";
15    PreparedStatement pst = con.prepareStatement(req2);
16    pst.setString(1, "FALL");
17    pst.setString(2, "Moussa");\textcolor[rgb]{1.00,0.00,0.00}{
18        int nbl = pst.executeUpdate();
19    } catch (SQLException e) {
20        e.printStackTrace();
21    } // les ressources Connection, PreparedStatement et ResultSet doivent être fermées
```

Listing 4 – Exemple de création et d'exécution de requêtes SQL paramétrées

Les requêtes paramétrées offrent plusieurs avantages :

- Leur traitement est plus rapide si elles sont exécutées plusieurs fois avec des paramètres différents;
- Elles améliorent la portabilité car les méthodes `setXXX` gèrent les conversions entre les types Java et ceux des SGBDs;
- Elles permettent d'éviter l'injection de codes SQL et ainsi d'améliorer la sécurité des données.

Remarque : L'**injection de codes SQL** est une faille de sécurité qu'un utilisateur malveillant peut exploiter en injectant une requête SQL non prévue par le développeur. Elle peut compromettre la sécurité de l'application.

Exemple :

Pour la requête suivante :

```
"SELECT * FROM utilisateur WHERE login = " + login + " and password = "
+ password + "";
```

Si un utilisateur saisit `' or ' = '` comme login et mot de passe, il aura accès à l'application !

4 Manipulation de résultats de requêtes SQL

Lorsqu’une requête SQL de consultation (de type select) est exécutée, en utilisant la méthode `executeQuery()`, elle retourne un objet de type `ResultSet`. Cet objet permet de parcourir toutes les lignes renvoyées par la requête, une à une, grâce à la méthode `next()`. Les méthodes `getXXX` (`XXX` désigne le type Java correspondant) de `ResultSet` permettent de récupérer les valeurs des colonnes de la table par leur index (`getInt(1)`, `getString(2)`) ou leur nom (`getString("prenom")`, `getInt("age")`).

```
1 String url = "jdbc:mysql://localhost:3306/club?serverTimezone=GMT";
2 //String url = "jdbc:postgresql://localhost:5432/club";
3 String user = "root"; String password = "";
4 //établissement d'une connexion à la base de données club
5 try (Connection connexion = DriverManager.getConnection(url, user, password)){
6     //création de requête SQL
7     Statement st = connexion.createStatement();
8     //retourne toutes les lignes de la table MEMBRE
9     ResultSet rs = st.executeQuery("SELECT * FROM MEMBRE");
10    // Récupère les lignes une à une
11    while(rs.next()){
12        String pnom = rs.getString("prenom");//ou rs.getString(2);
13        String nom = rs.getString("nom");
14        int age = rs.getInt("age");
15        System.out.println(pnom+" "+nom+" a "+age+" ans");
16    }
17 } catch (SQLException e) {
18     e.printStackTrace();
19 }// les ressources Statement et ResultSet doivent être fermées
```

Listing 5 – Exemple de manipulation de résultats d’une requête de consultation

La méthode `getMetaData()` de `ResultSet` renvoie un objet de type `ResultSetMetaData` permettant d’obtenir ses méta-données (nombre de colonnes, noms et types des colonnes, etc.).

```
1 //établissement d'une connexion à la base de données club
2 try (Connection connexion = DriverManager.getConnection(url, user, password)){
3     Statement st = connexion.createStatement();
4     ResultSet rs = st.executeQuery("SELECT * FROM MEMBRE");
5     //récupération des méta-données : colonnes et leurs types
6     ResultSetMetaData rmd = rs.getMetaData();
7     int nbCol = rmd.getColumnCount();
8     for(int i=1; i<=nbCol; i++){
9         System.out.print("Colonne: "+rmd.getColumnLabel(i));
10        System.out.println(" Type: "+rmd.getColumnTypeName(i));
11 } catch (SQLException e) {
12     e.printStackTrace();
13 }// les ressources Statement, ResultSet et ResultSetMetaData doivent être fermées
```

Listing 6 – Exemple de récupération de méta-données d’une requête

Un objet `ResultSet` doit être fermé lorsqu’il n’est plus utilisé. Il peut également être utilisé dans une instruction **try-with-resources**.

```
1 String url = "jdbc:mysql://localhost:3306/club?serverTimezone=GMT";
2 //String url = "jdbc:postgresql://localhost:5432/club";
3 String user = "root"; String password = "";
4 //établissement d'une connexion à la base de données club
5 try (Connection connexion = DriverManager.getConnection(url, user, password);
6     Statement st = connexion.createStatement();
7     ResultSet rs = st.executeQuery("SELECT * FROM MEMBRE")){
8     // Récupère les lignes une à une
9     while(rs.next()){
10        String pnom = rs.getString("prenom");//ou rs.getString(2);
11        String nom = rs.getString("nom");
12        int age = rs.getInt("age");
13    }
```

```

13     System.out.println(pnom+" "+nom+" a "+age+" ans");
14 }
15 } catch (SQLException e) {
16     e.printStackTrace();
17 } // fermeure automatique des ressources

```

Listing 7 – Exemple de manipulation de résultats d’une requête avec **try-with-resources**

L’exécution d’une requête de mise à jour (`executeUpdate`) renvoie le nombre de lignes mises à jour si l’opération s’est bien déroulée ou 0 en cas d’échec.

```

1 String url = "jdbc:mysql://localhost:3306/club?serverTimezone=GMT";
2 //String url = "jdbc:postgresql://localhost:5432/club";
3 String user = "root"; String password = "";
4 //établissement d'une connexion à la base de données club
5 try (Connection connexion = DriverManager.getConnection(url, user, password);
6     Statement st = connexion.createStatement()){
7     int nbl = st.executeUpdate("INSERT INTO MEMBRE VALUES ('id10', 'Modou', 'Diop',
8     42, 'Enseignant', 'M')");
9     if (nbl != 0)
10         System.out.println("Insertion réussie !!");
11     else
12         System.out.println("Problème d'insertion");
13 } catch (SQLException e) {
14     e.printStackTrace();
15 } // fermeure automatique des ressources

```

Listing 8 – Exemple de manipulation de résultats d’une requête de mise à jour